

Konzept und Aufbau

Verfügbarkeitsüberwachung mit Python, Prometheus und Grafana

Bleniii · github.com/bleniii/uptime-monitor · Stand September 2026

INHALT

- 1 Worum es geht
- 2 Zielsetzung und Leitgedanken
- 3 Anwendungsfälle und Abgrenzung
- 4 Architektur und Datenfluss
- 5 Metriken & Fehlerkategorisierung
- 6 Komponenten
- 7 Betrieb
- 8 Begriffe
- 9 Was der Aufbau belegt
- 10 Reflexion

1 Worum es geht

Ein praxisnahes Szenario

Eine Firma betreibt eine Website. Nachts um halb drei fällt sie aus, ein Dienst hängt, ein Zertifikat ist abgelaufen, der Server ist überlastet. Niemand merkt es. Am nächsten Morgen um 9:00 Uhr ruft ein Kunde an und sagt, die Seite gehe nicht. Zu diesem Zeitpunkt ist sie seit mehreren Stunden weg. Ein Kunde hat es gemerkt, bevor es die Firma gemerkt hat.

Genau diese Lücke schliesst ein Uptime-Monitor: ein Programm, das die überwachten Adressen in kurzen Abständen selbst aufruft, das Ergebnis festhält und daraus einen Verlauf über die Zeit erzeugt. Statt „die Seite geht nicht“ liegt dann eine belegte Aussage vor: seit wann, wie lange, mit welchem Fehlerbild und ob das Problem am Netzwerk, am Server oder an der Anwendung liegt.

Gebaut wurde das Werkzeug nicht, weil es solche Software nicht schon gäbe. Gebaut habe ich es, **um die vollständige Betriebskette einmal selbst zu durchlaufen: vom Skript über Tests, Systemdienst und Container bis zu Metriken, Zeitreihendatenbank und Dashboard.**

2 Zielsetzung und Leitgedanken

Der Monitor selbst ist bewusst klein gehalten. Der Gegenstand der Arbeit ist das, was um ihn herum liegt.

- Der Betrieb ist die Übung, nicht der Code. Ich wollte ein praxisnahes Szenario abbilden. Reproduzierbares Ausliefern, automatisiertes Prüfen, Betrieb ohne Aufsicht, Nachvollziehbarkeit von Änderungen und die Frage, woran man überhaupt merkt, dass etwas läuft.

- Eine Messung ist nur so gut wie ihre Eindeutigkeit. Ein Wert, der zwei Zustände gleich aussehen lässt, ist schlechter als kein Wert. Diese Erkenntnis zeigt sich in der Metrik-Semantik in Kapitel 5.
- Konfiguration gehört getrennt vom Code. Zu überwachende Ziele, Betriebsparameter und Datenquellen liegen in eigenen Dateien, nicht im Programmtext. Ausserdem sollten diese Dateien versioniert in einem Repository liegen, nicht nur auf dem laufenden System.
- Der Sollzustand wird beschrieben, nicht der Weg dorthin. Das gilt für den Compose-Stack ebenso wie für die Grafana-Provisionierung: Beide werden aus Dateien aufgebaut, die im Repository liegen, nicht aus Befehlen in einer Shell-History oder Klicks in einer Oberfläche.

3 Anwendungsfälle und Abgrenzung

Wofür das Werkzeug gedacht ist

Blackbox-Verfügbarkeitsprüfung. Geprüft wird von aussen, so wie ein Benutzer es täte: HTTP-Anfrage stellen, Antwort bewerten. Das ergänzt hostseitiges Monitoring, ersetzt es aber nicht.

Überschaubare Umgebungen. Ein Dienstleister mit einigen Dutzend Endpunkten, ein Team mit eigenen Diensten, eine Umgebung ohne bestehende Monitoring-Suite. Der Ressourcenbedarf ist gering, die Ziele stehen in einer Textdatei: *'targets.txt'*

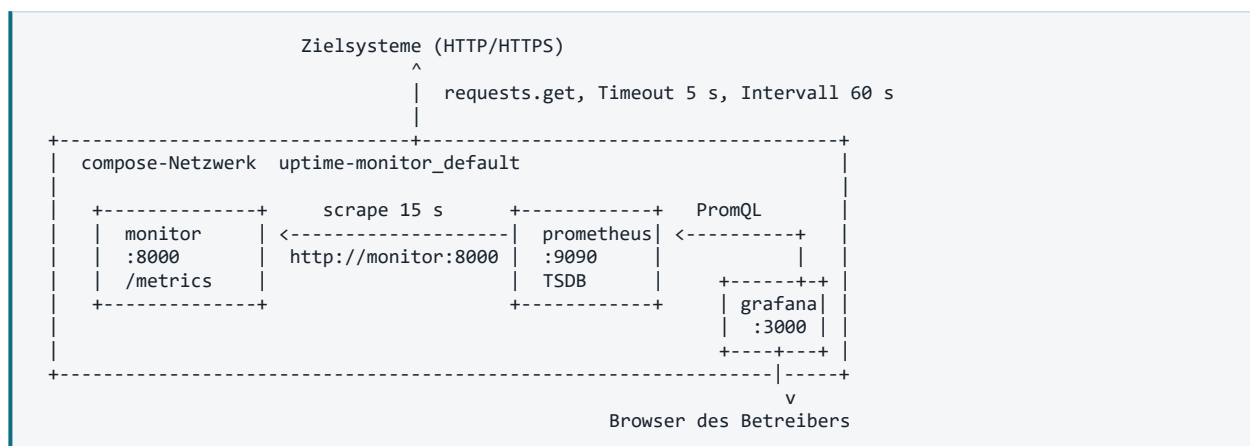
Nachweis von Reaktionszeiten. SLAs bestimmen bei einer Störung dass man innerhalb einer bestimmten Frist reagieren muss, hier liefert die Zeitreihe den Beleg, zusätzlich die Antwort auf die Frage, ob die Überwachung selbst zum fraglichen Zeitpunkt gelaufen ist.

Bewusste Abgrenzung

Der Prüfpunkt ist einer, nicht mehrere: Alle Anfragen gehen von einem Standort aus. Eine regional begrenzte Störung sieht damit aus wie eine globale. Geprüft wird der Statuscode, nicht der Inhalt — eine Seite, die eine leere Warenkorbansicht mit 200 ausliefert, gilt als gesund. Zertifikatslaufzeiten und Weiterleitungsketten werden nicht ausgewertet. Und Alarme entstehen in Prometheus, nicht als Anruf oder Nachricht; ein Alertmanager mit Eskalationswegen ist nicht Teil des Aufbaus.

4 Architektur und Datenfluss

Drei Dienste in einem gemeinsamen Compose-Netzwerk:



Der Monitor prüft die Ziele und stellt das Ergebnis im Prometheus-Expositionsformat unter `/metrics` bereit. Prometheus holt diesen Endpunkt alle 15 Sekunden ab und legt die Werte als Zeitreihen in seiner TSDB ab. Grafana fragt Prometheus per PromQL ab und stellt das Ergebnis dar.

Zwei Eigenschaften des Aufbaus sind dabei wesentlich:

Namensauflösung über Servicenamen. Compose legt ein eigenes Netzwerk an und betreibt darin einen internen DNS. Prometheus erreicht den Monitor als `monitor:8000`, Grafana erreicht Prometheus als `prometheus:9090`. Es sind keine IP-Adressen konfiguriert. Die veröffentlichten Ports dienen ausschliesslich dem Zugriff vom Host aus.

Zustand liegt in Volumes, Konfiguration liegt im Repository. Die Zeitreihendatenbank und der Grafana-Zustand liegen in Named Volumes und überleben einen Neustart. Die Konfigurationsdateien werden als Bind-Mount schreibgeschützt hineingereicht. Die Bauanleitung liegt damit im Repository, nicht im Container.

5 Metriken

Die drei Metriken

Metrik	Typ	Bedeutung
<code>uptime_erreichbar{url}</code>	Gauge	1 = Antwort erhalten, 0 = keine
<code>uptime_status{url}</code>	Gauge	HTTP-Statuscode der Antwort
<code>uptime_versuchsdauer_sekunden{url}</code>	Gauge	Dauer des Versuchs, auch im Fehlerfall

Alle drei tragen das Label `url`; Prometheus ergänzt beim Scrape `job` und `instance`. Jede Kombination aus Metrikname und Labels bildet eine eigene Zeitreihe.

Die entscheidende Trennung

`uptime_erreichbar` und `uptime_status` beantworten verschiedene Fragen. Ein Statuscode 404 bedeutet, dass der Server geantwortet hat, er ist also erreichbar, die Infrastruktur ist in Ordnung, das Problem liegt in der Anwendung. Ein Timeout dagegen liefert überhaupt keinen Statuscode.

Daraus folgen zwei getrennte Alarmbedingungen mit zwei verschiedenen Zuständen:

`uptime_erreichbar == 0` betrifft Netzwerk und Betrieb, `uptime_status >= 400` betrifft die Anwendung. Wer in der Früh darüber schaut, soll das aus der Meldung entnehmen können und nicht erst suchen müssen.

Warum die Versuchsdauer auch im Fehlerfall gemessen wird

Die Dauer ist bei einem Fehlerfall sehr wertvoll. Ein Wert nahe null bedeutet: Der Versuch ist sofort abgewiesen worden, es wurde gar nicht gewartet. Ein Wert am Timeout bedeutet: Die Verbindung stand, es kam nur nichts zurück, ein Hinweis auf Überlastung statt auf Abwesenheit. Dieselbe Fehlerklasse, zwei verschiedene Ursachen, unterscheidbar allein an der Zeit.

Fehlerkategorisierung

Fall	Erkennung	Stufe	Aussage
Erfolg	Statuscode < 400	INFO	Dienst antwortet erwartungsgemäss
HTTP-Fehler	Statuscode ≥ 400	WARNING	Server antwortet, Anwendung meldet einen Fehler
TIMEOUT	requests.exceptions.Timeout	ERROR	Verbindung stand, keine Antwort innerhalb 5 s
DNS ERROR	ConnectionError mit NameResolutionError als reason	ERROR	Name nicht auflösbar — falsche Adresse oder DNS gestört
CONNECTION ERROR	sonstige ConnectionError	ERROR	Host erreichbar, Port nimmt nichts an

Die Trennung von DNS- und Verbindungsfehler ist der Grund für das Auspacken der Ausnahme. `requests` verpackt die Ursache mehrschichtig in einer `ConnectionError`; ausgewertet wird das Attribut `reason` des innersten Objekts. Greift diese Auswertung nicht, etwa weil die Verpackung sich ändert, fällt der Fall auf `CONNECTION ERROR` zurück, statt eine Ausnahme im Fehlerbehandlungspfad auszulösen.

Vier Zeilen aus dem Log

```
19:24:06,495 INFO https://www.srf.ch/ Status 200 0.399s #1
19:24:06,540 ERROR http://diese-url-gibt-es-nicht.wvxyz DNS ERROR 0.045s #2
19:24:06,888 WARNING https://example.com/gibtesnicht Status 404 (HTTP FEHLER) 0.347s #3
19:24:06,890 ERROR http://127.0.0.1:9999 CONNECTION ERROR 0.002s #4
```

Zeile 1 und 3 haben ähnliche Laufzeiten: Eine Absage ist in etwa gleich schnell wie eine Zusage.

Zeile 2 und 4 - beide Male ist der Versuch gescheitert, bevor überhaupt eine Verbindung zustande kam. Aus vier Zeilen kann man ableiten, wo anzusetzen ist (**Frontend/Backend**) statt einfach einen Fehler zu sehen.

Keine veralteten Werte

Eine Gauge behält ihren zuletzt gesetzten Wert, bis ein neuer gesetzt wird. Fällt ein zuvor erreichbares Ziel aus, bliebe `uptime_status` deshalb auf dem letzten erfolgreichen Wert stehen und lieferte weiterhin eine 200, obwohl seit Stunden nichts mehr antwortet.

Ein ausbleibender Alarm sieht aus wie „alles in Ordnung“.

Der Fehlerpfad entfernt die Zeitreihe deshalb ausdrücklich über `.remove()`. Damit verschwindet die Zeile aus `/metrics`, und die Zeitreihe endet, statt mit einem alten Wert fortgeschrieben zu werden.

Existiert sie noch nicht, weil das Ziel von Beginn an nicht erreichbar war, wird der resultierende `KeyError` gezielt abgefangen. Gezielt, nicht mit einem breiten `except`, damit echte Fehler weiterhin sichtbar bleiben. Zurück bleibt eine Lücke in `uptime_status` und daneben eine ausdrückliche Aussage in `uptime_erreichbar`. Das ist eindeutig; ein stehengebliebener Wert wäre es nicht.

6 Komponenten

Bereich	Umsetzung
Anwendung	Python 3.11, requests, prometheus_client
Konfiguration	targets.txt, eine Adresse pro Zeile
Tests	pytest, unittest.mock
Betrieb	Container über docker compose; alternativ systemd-Unit
Paketierung	Docker, python:3.11-slim
Automatisierung	GitHub Actions
Erfassung	Prometheus
Darstellung	Grafana
Orchestrierung	docker compose

Prüf Schleife. `lade_targets()` liest die `targets.txt` (Ziel-URLs) ein und verwirft Leerzeilen. `pruefe_url()` führt eine einzelne Prüfung durch und liefert ein Ergebnis-Dictionary mit `status`, `dauer` und `fehler`. Die Hauptschleife durchläuft alle Ziele, protokolliert nach stdout und setzt die Metriken. Die Aufteilung ist die Voraussetzung dafür, dass die Logik überhaupt testbar ist.

Tests. Fünf pytest-Tests, vier davon mit Mock. `requests.get` wird durch ein Testobjekt ersetzt, das die gewünschte Antwort oder Ausnahme liefert. Die Tests laufen dadurch in Sekundenbruchteilen und ohne Netzwerkzugriff, sie prüfen die eigene Fehlerbehandlung, nicht die Verfügbarkeit fremder Server. Der Test für den DNS-Fall baut die verschachtelte Ausnahmestruktur nach, die `requests` im Ernstfall liefert.

Systemdienst. Eine systemd-Unit vom Typ `simple` mit `Restart=always`. Die Anwendung schreibt nach stdout; das Journal übernimmt Sammlung, Zeitstempel und Rotation. Ein Monitoring, das nach einem Neustart von Hand gestartet werden muss, ist nicht praktisch.

Container. Image auf Basis `python:3.11-slim`, Ausführung unter einem unprivilegierten Benutzer, `.dockerignore` zur Begrenzung des Build-Kontexts. Abhängigkeiten werden vor dem Anwendungscode installiert, damit die Layer-Zwischenspeicherung bei Codeänderungen greift.

Pipeline. GitHub Actions führt bei jedem Push und Pull Request die Tests aus und baut das Image. Geprüft wird damit nicht, ob es auf dem Entwicklungsrechner läuft — das ist bekannt —, sondern ob es auf einem System läuft, das nichts über diesen Rechner weiss. Das ist der Nachweis, dass alle Abhängigkeiten tatsächlich deklariert und alle nötigen Dateien tatsächlich versioniert sind.

Metrik-Endpunkt. `prometheus_client` stellt über `start_http_server(8000)` einen HTTP-Endpunkt unter `/metrics` bereit, der die aktuellen Werte im Expositionsformat ausgibt. Zusätzlich liefert die Bibliothek Prozessmetriken zu Speicher, Laufzeit und Garbage Collection.

Compose-Stack. Eine `docker-compose.yml` beschreibt alle drei Dienste. Der Monitor wird aus dem lokalen Dockerfile gebaut, Prometheus und Grafana kommen als Images aus der Registry. Konfigurationsdateien werden schreibgeschützt eingehängt, Zustandsdaten liegen in Named Volumes.

Erfassung. Ein `scrape_config` mit einem statischen Ziel und einem Intervall von 15 Sekunden. Das Prüfintervall der Anwendung von 60 Sekunden ist davon unabhängig; zwischen zwei Messungen liefert der Endpunkt denselben Wert mehrfach aus. Im Verlauf zeigt sich das als Treppe, deren Stufenbreite dem Prüfintervall entspricht.

Dashboard. Drei Panels, jedes mit einer bewussten Darstellungsentscheidung. Die Verfügbarkeit als State timeline. Die Zustände sind über Value Mappings beschriftet. Die Antwortzeit als Time series mit `avg_over_time(...[5m])`, was die Treppenstufen glättet, und mit der Einheit Sekunden, damit die Achse selbsterklärend beschriftet ist. Der Statuscode als Time series mit 'Interpolation Step after' und einem Schwellwert bei 400. Eine lineare Interpolation würde zwischen 200 und 404 Zwischenwerte zeichnen, die nie gemessen wurden.

Die Grafana-Datenquelle wird beim Start aus einer YAML-Datei geliefert und ist in der Oberfläche nicht editierbar. Sie verweist mit `access: proxy` auf `http://prometheus:9090`, sodass die Abfrage vom Grafana-Server ausgeht und nicht vom Browser — der Servicename existiert nur im internen Netzwerk.

7 Betrieb

```
docker compose up -d --build
docker compose ps
```

Dienst	Adresse	Funktion
monitor	<code>http://localhost:8000/metrics</code>	Erhebt und exponiert die Messwerte
prometheus	<code>http://localhost:9090</code>	Erfasst und speichert die Zeitreihen
grafana	<code>http://localhost:3000</code>	Stellt sie dar

Der Compose-Stack ist die vorgesehene Betriebsart; die `systemd`-Unit unter `systemd/` ist die Alternative für Umgebungen ohne Container-Laufzeit. Beide starten dasselbe Programm auf Port 8000 des Hosts, und ein Port lässt sich nur einmal belegen, die laufende Variante muss deshalb gestoppt sein, bevor die andere startet.

Die Unit ist entstanden, bevor der Container dazukam, und bewusst erhalten geblieben: Sie zeigt denselben Dienst einmal als Prozess unter `systemd` und einmal als Container, mit denselben Anforderungen an Neustartverhalten und Logging.

8 Begriffe

Begriff	Bedeutung
Blackbox-Monitoring	Prüfung von aussen, ohne Kenntnis der inneren Vorgänge — so wie ein Benutzer den Dienst erlebt.
Statuscode	Zahl, die ein Webserver mit jeder Antwort mitschickt. 200 in Ordnung, 404 nicht gefunden, 500 Fehler auf dem Server.
DNS	Namensdienst des Internets. Übersetzt Namen wie example.com in die Adresse, unter der der Rechner erreichbar ist.
Metrik	Ein Messwert als Zahl, den eine Auswertungssoftware regelmässig abholt.
Gauge	Metriktyp für Werte, die steigen und fallen können — im Gegensatz zum Counter, der nur wächst.
Label	Zusatzangabe an einer Metrik, hier die geprüfte Adresse. Erzeugt pro Ausprägung eine eigene Zeitreihe.
Zeitreihe	Eine Folge von Wertepaaren aus Zeitstempel und Zahl.
Scrape	Der regelmässige Abruf des Metrik-Endpunkts durch Prometheus.
PromQL	Abfragesprache von Prometheus, mit der Zeitreihen gefiltert und berechnet werden.
TSDB	Zeitreihendatenbank; auf Zeitstempel-Wert-Paare spezialisierter Speicher.
Container	Paket aus Anwendung und vollständiger Laufzeitumgebung. Verhält sich unabhängig vom Wirtssystem gleich.
Volume	Von Docker verwalteter Speicherbereich, der unabhängig vom Container fortbesteht.
Bind-Mount	Einhängen einer Datei oder eines Verzeichnisses vom Wirtssystem in den Container.
Provisionierung	Einrichtung aus Konfigurationsdateien beim Start statt durch Klicks in der Oberfläche.
Pipeline	Automatische Prüfstrecke, die bei jeder Änderung Tests ausführt und das Paket baut.
Repository	Versionierte Ablage aller Dateien mit vollständiger Änderungsgeschichte.

9 Was der Aufbau belegt

- **Python** mit differenzierter Fehlerbehandlung und funktionaler Zerlegung
- **Testautomatisierung** mit pytest und Mocking, einschliesslich nachgebauter Ausnahmestrukturen
- **Linux-Betrieb** als systemd-Dienst mit Neustartverhalten und Journal-Anbindung
- **Containerisierung** mit unprivilegierter Ausführung und bewusstem Layer-Aufbau
- **CI/CD** mit automatischer Prüfung und Image-Build bei jeder Änderung
- **Observability** mit selbst definierten Metriken, Prometheus-Erfassung und PromQL
- **Infrastructure as Code** über deklarativen Compose-Stack und provisionierte Datenquellen
- **Verifikation** als eigenständige Fähigkeit: Verhalten im Vorher-Nachher-Vergleich nachweisen, statt es anzunehmen
- **Dokumentation** von Konzept, Code und technischer Anleitung

10 Reflexion

Mein beruflicher Hintergrund liegt im Microsoft-Umfeld: als System-Administrator. Mit DevOps und Infrastructure as Code hatte ich bis zu diesem Projekt sehr wenige Berührungspunkte. Die Begriffe waren mir geläufig, die Zusammenhänge dahinter nicht.

Genau das hat sich hier geändert. Der grösste Gewinn liegt nicht im fertigen Werkzeug, sondern darin, dass eine Reihe von Themen, die vorher Fremdwörter waren, jetzt einen konkreten Bezugspunkt haben. Ich kann beispielsweise nicht nur sagen, was eine Pipeline ist, sondern habe eine gebaut und gesehen, was sie prüft und was nicht.

Drei Themen haben mich dabei besonders interessiert:

- **Automatisierung:** Der Unterschied zwischen „ich habe es eingerichtet“ und „es richtet sich selbst ein“ wurde erst greifbar, als ich beides nacheinander gemacht habe.
- **Containerisierung und ihre Zusammenhänge:** Was ein Container tatsächlich einschliesst, wie Netzwerk und Namensauflösung darin funktionieren, warum localhost im Container etwas anderes bedeutet als auf dem Host, darüber kann man lesen, aber man versteht es erst richtig, wenn man beim bauen und konfigurieren, Fehler macht und diese korrigiert.
- **Reproduzierbarkeit.** In meinem bisherigen Alltag habe ich Änderungen im Admin Center geklickt oder per PowerShell abgesetzt. Beides funktioniert, aber danach ist nur das Ergebnis vorhanden, nicht der Weg dorthin. In diesem Projekt steht der Sollzustand in einer Datei, die versioniert ist: nachlesbar, überprüfbar, wiederholbar.
- **Linux.** Systemdienste mit systemd, Logging über das Journal, Prozesse und Ports, Benutzerrechte und Paketverwaltung. Vieles davon habe ich erst verstanden, als es nicht funktioniert hat. Dass ein Dienst z. B. *enabled* sein kann, ohne *active* zu sein, oder dass ein belegter Port eine Fehlermeldung erzeugt, die nach einem Konfigurationsfehler aussieht.